

Introduction to ROS – Autonomous Driving

Team 16: Miao Kai, Huang Gengxiang, Mu Junhao, Bi Yunfei, Chen Yiwu

1. Introduction

In this project, we implement an autonomous driving system using the Robot Operating System (ROS). Our objective is to navigate a predefined route in a simulated urban environment efficiently, without collisions, and in full compliance with traffic rules. The system integrates real-time perception, planning, decision-making, and vehicle control.

This documentation is structured as follows: Section 2 outlines the overall system architecture and inter-module communication. Section 3 provides a detailed explanation of each core module. Section 4 summarizes the project and discusses potential improvements.

2. Overall Structure

Our system is structured into five interconnected modules as shown in Figure 1:

- **Simulation:** Acts as the environment and provides semantic data (e.g. object labels, traffic lights, and poses).
- **Perception:** Receives semantic camera data from Simulation and publishes safety-relevant signals, such as obstacle warnings and traffic light states.
- **OdometryPub:** Converts pose data from Simulation into ROS-standard odometry messages for downstream use in planning.
- **Planning:** Subscribes to goal and odometry information and generates velocity commands for navigation.
- **State Machine:** Receives information from Perception and Planning to determine whether to stop, slow down, or proceed normally.
- **Control:** Receives modified velocity commands from the State Machine and converts them into steering/throttle/brake commands for execution.

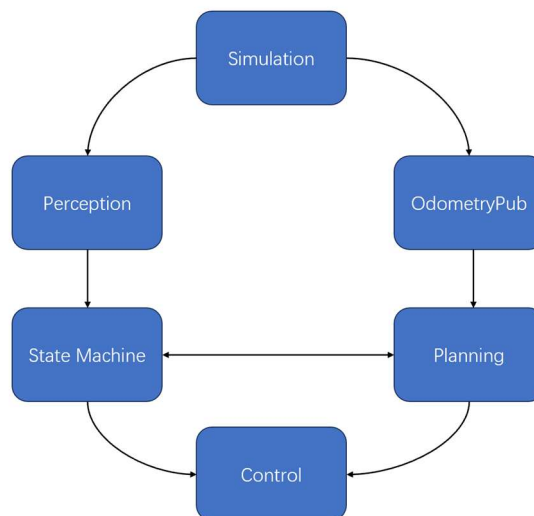


Figure 1. Module communication diagram of the autonomous driving system.

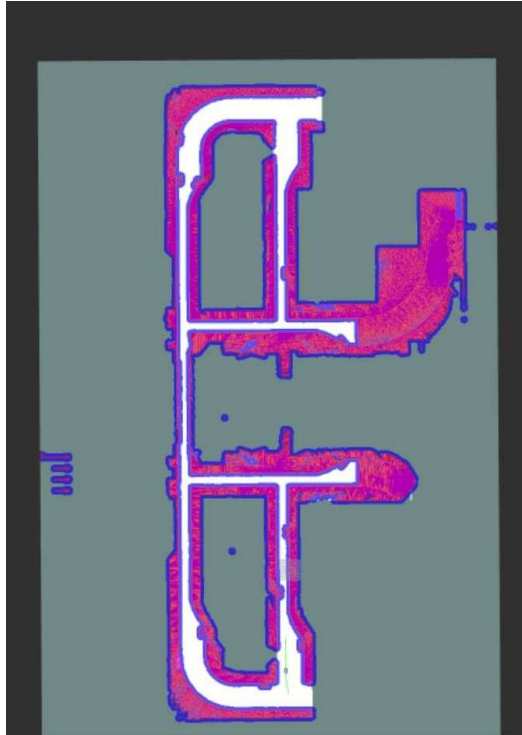


Figure 3. Visualization of the simulated urban driving environment.

3. Main Components

3.1 Perception

The perception module is implemented in the `auto_car_perception` package. It consists of the following key components:

- **Obstacle Detection:**
 - Subscribes to semantic camera data and detects obstacles based on object types and distances.
 - Uses configurable parameters to determine whether an obstacle is close enough to trigger either a stop (`/obstacle_stop`) or a slow-down (`/obstacle_slow`) condition.
 - Performs geometric filtering and prioritizes obstacles in front of the vehicle.
- **Traffic Light Recognition:**
 - Processes label information from the semantic camera to identify visible traffic lights.
 - Determines their states and visibility range.
 - Publishes a Boolean message to `/stop` when a red light is detected and in the field of view.

The perception module provides safety-critical information to the state machine for real-time driving decisions.

3.2 Planning

The planning functionality is supported by tools in the `auto_car_utils` package. It includes the

following nodes:

- **Goal Publisher:**
 - Publishes a fixed goal pose using `geometry_msgs::PoseStamped`, which serves as the target for the global planner.
 - Can be configured through launch files to place the goal in different parts of the map.
- **Odometry Publisher:**
 - Converts pose and velocity information from the simulation into `nav_msgs::Odometry` messages.
 - Maintains consistent update rates and transforms for planner input.
 -

These planning utilities support stable and consistent generation of velocity commands used for downstream control.

Figure 4 illustrates the internal workflow of the planning utilities. The system continuously broadcasts vehicle odometry from simulation TF frames, while the goal publisher sequentially issues navigation goals and cancels them if necessary based on safety signals.

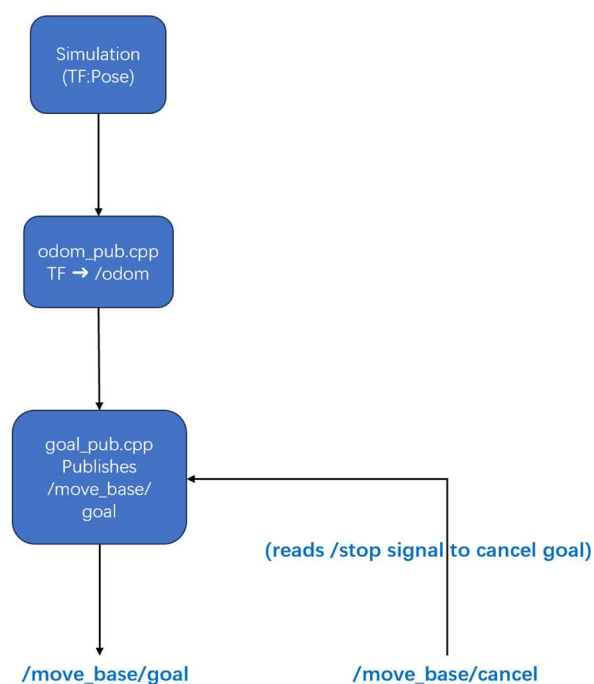


Figure 4 Planning logic diagram

3.3 Decision Making

The decision-making component is implemented in the `state_machine` package and manages behavior switching through a finite state machine. Its functionality includes:

- **Safety Condition Aggregation:**
 - Subscribes to multiple safety-related topics: `/stop`, `/traffic_light_presence`, `/obstacle_stop`, `/obstacle_slow`, and `/cmd_vel`.
 - Consolidates inputs to determine an appropriate response for the vehicle's

current context.

- **State Machine Logic:**
 - Maintains internal logic with three primary states:
 - **NORMAL:** Default driving behavior using unmodified `/cmd_vel`.
 - **SLOW_DOWN:** Reduces linear velocity proportionally when an obstacle or traffic light is near.
 - **STOP:** Sets linear velocity to zero when a red light or hard obstacle is encountered.
- **Safe Command Output:**
 - Publishes the selected action as `/safe_cmd_vel` for use by the vehicle controller.
 - Ensures safety takes precedence over planning behavior.

This modular logic enables the vehicle to behave cautiously in complex driving scenarios. A visual representation of the state machine logic is shown in **Figure 5**. The vehicle behavior switches between **NORMAL**, **SLOW_DOWN**, and **STOP** based on safety-related signals such as traffic lights and obstacles. Transitions between states are governed by clearly defined Boolean conditions. The state machine prioritizes **STOP** over **SLOW_DOWN** and **NORMAL** to ensure safety-critical actions are handled first.

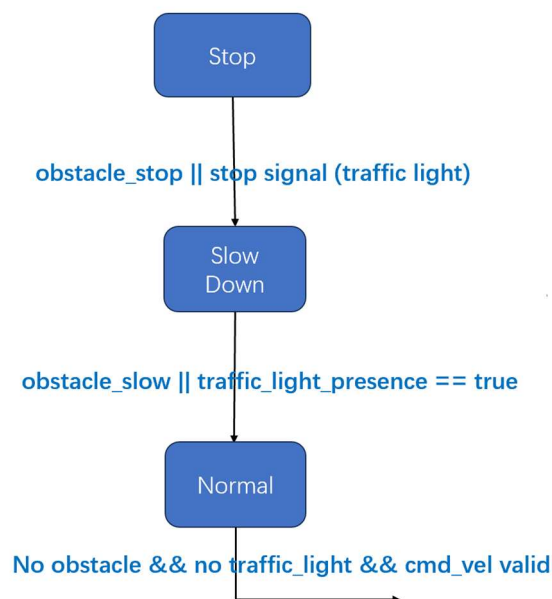


Figure 5. A visual representation of the state machine logic

Additionally, the state machine publishes a custom message of type `StateStatus` that encapsulates the current state (**STOP**, **SLOW_DOWN**, **NORMAL**) and the corresponding Boolean trigger flags. This message allows external modules or visualization tools to monitor the internal decision status of the FSM in a compact and structured format. It improves debugging and system observability during simulation runs.

The control module is implemented in the `auto_car_utils` package, with core logic in `twist2vc.cpp`. It performs:

- This approach guarantees safe driving while preserving steering behavior from the original path planner.

Figure 6 provides a detailed view of the control module. It separates longitudinal control (throttle/brake) and lateral control (steering) based on independent velocity command topics. This modular design ensures that the system remains reactive to safety overrides while retaining smooth steering trajectories.

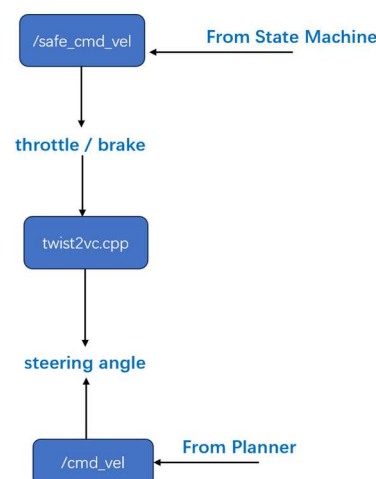


Figure 6. Control module logic

This project demonstrates a complete autonomous driving pipeline in a simulated ROS environment. We successfully implemented perception (obstacle and traffic light detection), motion planning utilities and a controller, decision logic through a state machine, and vehicle

control conversion.

Our system responds effectively to road scenarios, including traffic signals and path tracking. While functional, further enhancements—such as dynamic obstacle avoidance and smoother acceleration profiles—could improve performance in more complex settings. The modular design also allows easy extension for real-world integration or more advanced algorithms.

Note: In extremely rare cases, the vehicle may collide with the road edge due to initialization delays or minor simulation timing issues. This behavior is highly uncommon — in the vast majority of runs, the system operates reliably without issues. If such a case occurs, restarting the project will typically resolve it.